

## DESCRIPTION D'UNE MISSION | BTS SIO

|                     |                     |
|---------------------|---------------------|
| <b>Prénom – Nom</b> | RAPHEHISON Mandresy |
| <b>N° mission</b>   | E-02                |

|                  |                                    |                                                |
|------------------|------------------------------------|------------------------------------------------|
| <b>Option</b>    | <input type="checkbox"/> SISR      | <input checked="" type="checkbox"/> SLAM       |
| <b>Situation</b> | <input type="checkbox"/> Formation | <input checked="" type="checkbox"/> Entreprise |

|                                |                                                                             |                                                                                     |
|--------------------------------|-----------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| <b>Lieu de réalisation</b>     | Carrefour CSI – 93 Avenue de Paris, 91300 Massy                             |  |
| <b>Période de réalisation</b>  | <b>Du :</b> 01/2025 <b>Au :</b> 02/2026                                     |                                                                                     |
| <b>Modalité de réalisation</b> | <input checked="" type="checkbox"/> VÉCUE <input type="checkbox"/> OBSERVÉE |                                                                                     |

|                                |                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Intitulé de la mission</b>  | Développement d'un script Java pour récupérer les métriques Dynatrace                                                                                                                                                                                                                                                                                                                                                                |
| <b>Description du contexte</b> | <p>Dans le cadre du projet MQ APIM, certaines métriques nécessaires au suivi de l'infrastructure n'étaient pas disponibles nativement dans les datasources déjà configurées.</p> <p>C'était notamment le cas pour les métriques CPU/RAM des pods Kubernetes APIM et l'état du trafic HTTP sur les domaines Apigee, disponibles uniquement via l'API Dynatrace. Il a donc fallu développer un mécanisme de collecte personnalisé.</p> |

|                                      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|--------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Ressources et outils utilisés</b> | <ul style="list-style-type: none"> <li>- Eclipse IDE intégré à la plateforme LOGM</li> <li>- Java (framework LOGM : Schedule, WebUtils, TypedMap)</li> <li>- API Dynatrace V2 (endpoint metrics/query)</li> <li>- Token d'authentification Dynatrace fourni par le responsable Dynatrace de l'équipe</li> <li>- Base de données InfluxDB (cible de stockage des métriques)</li> <li>- Grafana (visualisation finale)</li> <li>- Documentation Dynatrace API V2 et documentation interne LOGM</li> </ul> |
| <b>Résultat attendu</b>              | <ul style="list-style-type: none"> <li>- Un script Java planifié toutes les 5 minutes interrogeant l'API Dynatrace</li> <li>- Les métriques collectées (CPU pods, trafic HTTP) stockées dans InfluxDB</li> <li>- Un panel Grafana affichant ces métriques avec seuils d'alerte configurés</li> </ul>                                                                                                                                                                                                    |
| <b>Contraintes</b>                   | <ul style="list-style-type: none"> <li>- Données non disponibles nativement : nécessité de développer un script sur mesure</li> <li>- Authentification par token Dynatrace à obtenir auprès de l'équipe responsable</li> <li>- Fréquence d'exécution d'une minute imposée pour avoir des données quasi temps réel</li> <li>- Intégration dans l'environnement LOGM avec ses classes et conventions propres</li> </ul>                                                                                   |

|                                             |                                                                                                                                                 |
|---------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Compétences associées (voir tableau)</b> | <ul style="list-style-type: none"> <li>- B2.1 – Concevoir et développer une solution applicative</li> <li>- B2.3 – Gérer les données</li> </ul> |
|---------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|

### Description des étapes de réalisation (démarche, méthodes, techniques utilisées)

#### 1. IDENTIFICATION DU BESOIN ET ANALYSE DE L'API DYNATRACE

Les métriques CPU et RAM des pods Kubernetes APIM ainsi que l'état du trafic HTTP sur les domaines Apigee étaient disponibles dans Dynatrace mais non exposées via une datasource compatible avec LOGM. J'ai consulté la documentation Dynatrace API V2 pour identifier l'endpoint metrics/query et comprendre la structure de la réponse JSON.

Un token d'authentification a été demandé au responsable Dynatrace de l'équipe. Ce token est transmis dans le header HTTP Authorization de chaque requête.

## 2. DÉVELOPPEMENT DU SCRIPT JAVA

Le script a été développé dans l'IDE Eclipse intégré à LOGM. Il s'appuie sur le framework LOGM en étendant la classe Schedule et en utilisant l'annotation `@LogmSchedule(every = "5m")` pour définir sa fréquence d'exécution.

La méthode `onSchedule()` constitue le cœur du script :

### Étape 1 — Appel HTTP vers l'API Dynatrace

Les headers d'authentification sont construits avec le token Dynatrace :

Authorization: Api-Token <token>

Accept: application/json

La requête est envoyée via `WebUtils.invoke()` avec un timeout de 60 secondes.

L'URL cible l'endpoint `metrics/query` avec un `metricSelector` filtrant les pods

Kubernetes du namespace APIM ciblé (`CLOUD_APPLICATION_NAMESPACE-9C2183A0ED9A120B`).

### Étape 2 — Parsing de la réponse JSON

La réponse est désérialisée en `Map<String, Object>`. Le champ 'result' contient une liste de métriques. Pour chacune, on extrait :

- `metricId` : identifiant de la métrique
- `dimensionMap` : namespace Kubernetes associé
- `timestamps` et `values` : séries temporelles des valeurs mesurées

### Étape 3 — Envoi vers InfluxDB

Chaque point de mesure (`timestamp` + `valeur` + `namespace`) est formaté et envoyé vers la base InfluxDB de LOGM pour être stocké en série temporelle.

## 3. CONFIGURATION DU PANEL GRAFANA

Une datasource InfluxDB a été configurée dans Grafana pointant vers la base alimentée par le script. Un panel de supervision a été créé pour afficher l'utilisation CPU des pods APIM. Des seuils d'alerte (`thresholds`) ont été configurés selon les besoins exprimés par le client, par exemple un seuil critique à 10% d'utilisation CPU.

|                              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Conclusion</b>            | Cette mission m'a permis de développer une solution complète de collecte de données, de l'appel API jusqu'à la visualisation dans Grafana. Le développement from scratch dans le framework LOGM m'a confronté à ses conventions et ses classes utilitaires, ce qui a renforcé ma compréhension de l'architecture interne de la plateforme. La mise en place du pipeline Dynatrace → Java → InfluxDB → Grafana m'a également donné une vision concrète de l'intégration de sources de données hétérogènes. |
| <b>Productions associées</b> | <ul style="list-style-type: none"><li>- Script Java <code>MySchedule.java</code> intégré dans le dépôt LOGM (GitLab interne)</li><li>- Métriques Dynatrace stockées dans InfluxDB, accessibles via Grafana</li><li>- Panel CPU pods APIM avec seuils d'alerte dans le dashboard MQ APIM</li></ul>                                                                                                                                                                                                         |